Advanced Web Application Development Technologies

Nikolaos Korfiatis & Dimitris Ringas

nkorf@ionio.gr | riggas@ionio.gr

Department of Informatics

Ionian University



HTTP and Python

Web Technologies & API Calls



Session Objectives

- **Understanding** the operation and importance of the HTTP protocol
- **Familiarization** with HTTP methods, status codes and headers
- **Practical application** of HTTP calls with Python
- **Handling** errors and timeouts
- **Processing** JSON payloads in requests and responses
- **Developing** skills for consuming REST APIs

What is HTTP?

The **HyperText Transfer Protocol (HTTP)** is the fundamental protocol of the World Wide Web:

- **Application Layer Protocol** in the TCP/IP stack
- **Client-Server Architecture** - request and response
- **Stateless Protocol** - each request is independent
- **Text-based** - human readable format
- **Port 80** (HTTP) and **Port 443** (HTTPS)

Why is it important?

- Basis of all web applications
- Foundation for REST APIs
- Communication between microservices
- Data exchange in distributed systems

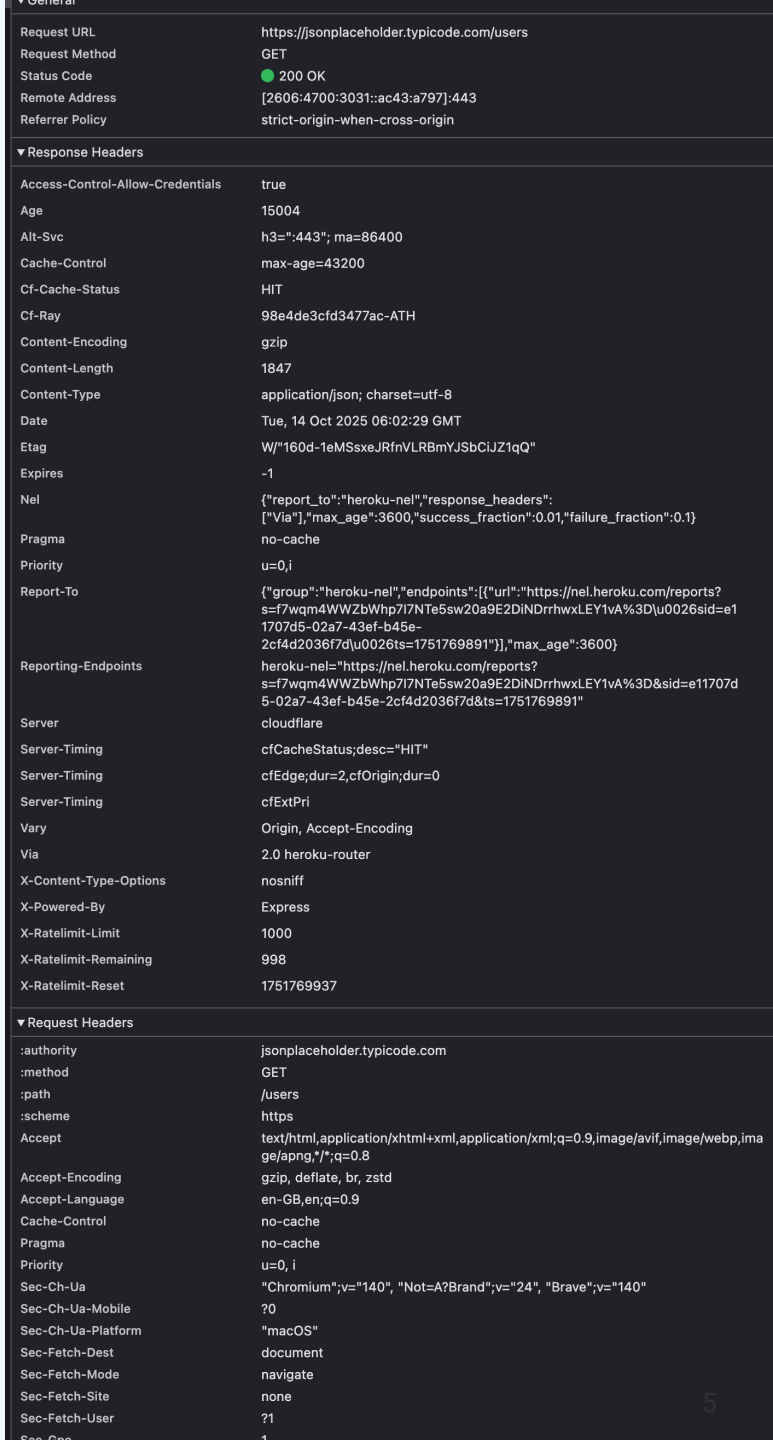
HTTP Request Structure

```
GET /api/users/123 HTTP/1.1
Host: api.example.com
User-Agent: Mozilla/5.0
Accept: application/json
Authorization: Bearer token123
Content-Type: application/json

{
  "update": "some data"
}
```

Components of an HTTP Request:

1. **Request Line**: Method + URL + HTTP Version
2. **Headers**: Metadata about the request
3. **Body**: Data (optional, depending on method)



▾ General

Request URL	https://jsonplaceholder.typicode.com/users
Request Method	GET
Status Code	200 OK
Remote Address	[2606:4700:3031::ac43:a797]:443
Referrer Policy	strict-origin-when-cross-origin

▾ Response Headers

Access-Control-Allow-Credentials	true
Age	15004
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	98e4de3cfd3477ac-ATH
Content-Encoding	gzip
Content-Length	1847
Content-Type	application/json; charset=utf-8
Date	Tue, 14 Oct 2025 06:02:29 GMT
Etag	W/"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Priority	u=0,i
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=f7wqm4WWZbWhp7I7NTe5sw20a9E2DiNDrrhwxLEY1vA%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1751769891"}],"max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=f7wqm4WWZbWhp7I7NTe5sw20a9E2DiNDrrhwxLEY1vA%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1751769891"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=2,cfOrigin;dur=0
Server-Timing	cfExtPri
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	998
X-Ratelimit-Reset	1751769937

▾ Request Headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/users
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-GB,en;q=0.9
Cache-Control	no-cache
Pragma	no-cache
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="140", "Not=A?Brand";v="24", "Brave";v="140"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"macOS"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1

HTTP Methods (Verbs)

Method	Purpose	Body	Idempotent*
GET	Retrieve data	No	Yes
POST	Create new	Yes	No
PUT	Full update	Yes	Yes
PATCH	Partial update	Yes	No
DELETE	Delete	No	Yes
HEAD	Headers only	No	Yes
OPTIONS	Allowed methods	No	Yes

Mapping to CRUD Operations:

- **Create** → POST
- **Read** → GET
- **Update** → PUT/PATCH
- **Delete** → DELETE

“ * An HTTP method is idempotent when executing it repeatedly has the same effect as executing it once.
⇒ If we send the same HTTP request multiple times, the server state does not change any further after the first call. ”

HTTP Status Codes

1xx Informational

- `100 Continue` - Continue the request

2xx Success

- `200 OK` - Successful request
- `201 Created` - New resource created
- `204 No Content` - Success with no content

3xx Redirection

- `301 Moved Permanently` - Permanent move
- `302 Found` - Temporary move

Example Response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 85

{
  "id": 123,
  "name": "John Doe",
  "email": "john@example.com"
}
```

HTTP Status Codes (continued)

5xx Server Error

- `500 Internal Server Error` - Generic server error
- `502 Bad Gateway` - Problem at the gateway
- `503 Service Unavailable` - Service not available
- `504 Gateway Timeout` - Timeout at the gateway

4xx Client Error

- `400 Bad Request` - Wrong syntax
- `401 Unauthorized` - Authentication required
- `403 Forbidden` - Access not allowed
- `404 Not Found` - Resource not found
- `429 Too Many Requests` - Rate limiting

HTTP Headers - Main Categories

Request Headers

- `Host: api.example.com` - Server hostname
- `User-Agent: MyApp/1.0` - Client information
- `Accept: application/json` - Preferred content types
- `Authorization: Bearer token123` - Authentication
- `Content-Type: application/json` - Body format

Response Headers

- `Content-Type: application/json` - Response format
- `Content-Length: 256` - Body size in bytes
- `Server: nginx/1.18.0` - Server software
- `Cache-Control: max-age=3600` - Caching instructions

Custom Headers

- `X-API-Key: your-api-key`
- `X-Request-ID: uuid-here`
- `X-Rate-Limit-Remaining: 99`

Python and HTTP



Python and HTTP - Introduction

Alternative options for HTTP calls in python:

Built-in Libraries

- `urllib.request` - Standard library
- `http.client` - Low-level HTTP client

Third-party Libraries

- **`requests`** - Human-friendly HTTP library
- `httpx` - Async HTTP client
- `aiohttp` - Asynchronous HTTP client/server

Why `requests`?

- **Simple API** - easy to use
- **Feature-rich** - complete feature set
- **Well-documented** - excellent documentation
- **Community support** - large community

Example request in python:

```
import requests
response = requests.get('https://api.github.com/users/octocat')
```

Python Environment Setup

1. Python Installation

```
# macOS (with Homebrew)
which python3  # If not found.. https://www.python.org/downloads/ or
brew install python3

# Ubuntu/Debian
sudo apt update && sudo apt install python3 python3-pip

# Verify installation
python3 --version
pip3 --version
```

2. Virtual Environment Setup

```
# Create virtual environment
python3 -m venv http_workshop
cd http_workshop

# Activate (macOS/Linux)
source bin/activate

# Activate (Windows)
Scripts\activate.bat
```

3. Install Required Packages

```
pip install requests beautifulsoup4 pandas matplotlib
```

The requests library - Basic Usage

Simple GET Request

```
import requests

# Basic GET request
response = requests.get('https://httpbin.org/json')

# Check status code
if response.status_code == 200:
    data = response.json()
    print("Success:", data)
else:
    print("Error:", response.status_code)
```

Response Object Properties

```
print(f"Status: {response.status_code}")
print(f"Headers: {response.headers}")
print(f"Content-Type: {response.headers['content-type']}")
print(f"Text: {response.text}")
print(f"JSON: {response.json()}")  # If the response is JSON
print(f"Time: {response.elapsed}")
```

GET Requests with Parameters

Query Parameters

```
import requests

# Method 1: URL string
url = "https://httpbin.org/get?name=John&age=30"
response = requests.get(url)

# Method 2: params dictionary (preferred way)
params = {
    'name': 'John',
    'age': 30,
    'city': 'Athens'
}
response = requests.get('https://httpbin.org/get', params=params)
print("Final URL:", response.url)
```

Headers Customization

```
headers = {
    'User-Agent': 'MyPythonApp/1.0',
    'Accept': 'application/json',
    'Authorization': 'Bearer your-token-here'
}
response = requests.get('https://api.github.com/user', headers=headers)
```

POST Requests and Data Sending

Sending JSON Data

```
import requests
import json

# Method 1: Using json parameter (preferred)
data = {
    'name': 'Nikos Papadopoulos',
    'email': 'nikos@example.com',
    'age': 25
}
response = requests.post('https://httpbin.org/post', json=data)

# Method 2: Manual JSON encoding
json_data = json.dumps(data)
headers = {'Content-Type': 'application/json'}
response = requests.post('https://httpbin.org/post',
                         data=json_data, headers=headers)
```

Form Data

```
# application/x-www-form-urlencoded
form_data = {'username': 'user123', 'password': 'secret'}
response = requests.post('https://httpbin.org/post', data=form_data)
```

Error Handling and Exceptions

HTTP Error Handling

```
import requests
from requests.exceptions import RequestException

try:
    response = requests.get('https://httpbin.org/status/404',
                            timeout=5)

    # Check for HTTP errors (4xx and 5xx)
    response.raise_for_status()

    # Process successful response
    data = response.json()
    print("Success:", data)

except requests.exceptions.HTTPError as e:
    print(f"HTTP Error: {e}")
    print(f"Status Code: {response.status_code}")

except requests.exceptions.ConnectionError:
    print("Connection Error - Check your internet connection")

except requests.exceptions.Timeout:
    print("Request timed out")

except requests.exceptions.RequestException as e:
    print(f"General Request Error: {e}")
```

Timeouts and Performance

Timeout Configuration

```
import requests

# Single timeout (for all phases)
response = requests.get('https://httpbin.org/delay/2', timeout=5)

# Tuple timeout: (connect, read)
response = requests.get('https://httpbin.org/delay/2',
                        timeout=(3, 10))

# No timeout (dangerous!)
# response = requests.get(url)  # Avoid this!
```

Timeouts and Performance

Session Objects for Performance

```python
import requests

# Connection reuse
with requests.Session() as session:
    session.headers.update({'User-Agent': 'MyApp/1.0'})

    # Multiple requests with the same session
    response1 = session.get('https://httpbin.org/headers')
    response2 = session.get('https://httpbin.org/user-agent')
    response3 = session.get('https://httpbin.org/json')
```

- When we use plain requests.get() or requests.post() calls, each call performs the following steps:
  - Opens a new TCP connection to the server.
  - Performs a TLS handshake (if HTTPS).
  - Sends the request.
  - Receives the response.
  - Closes the connection.
- When we use a Session object, we tell Python to reuse the same TCP connection across requests:
  - A pool of active connections is created for each host.
  - Connections are not closed after each request, but kept alive.
  - Cookies and headers remain stored in the session.
  - Requests are much faster, since repeated connection setup is avoided.

JSON Processing with Python

JSON Parsing

```
import requests
import json

response = requests.get('https://jsonplaceholder.typicode.com/users/1')

# Automatic JSON decoding
user_data = response.json()

print(f"Name: {user_data['name']}")
print(f"Email: {user_data['email']}")
print(f"Company: {user_data['company']['name']}")

# Manual JSON handling for complex cases
try:
    json_text = response.text
    data = json.loads(json_text)
except json.JSONDecodeError as e:
    print(f"Invalid JSON: {e}")
```

Nested JSON Navigation

```
# Safe access to nested data
def safe_get(data, *keys):
    for key in keys:
        if isinstance(data, dict) and key in data:
            data = data[key]
        else:
            return None
    return data

company_name = safe_get(user_data, 'company', 'name')
geo_lat = safe_get(user_data, 'address', 'geo', 'lat')
```

✋ That's enough.. *for today*!

Continue at:

https://github.com/riggas-ionio/web-II/

